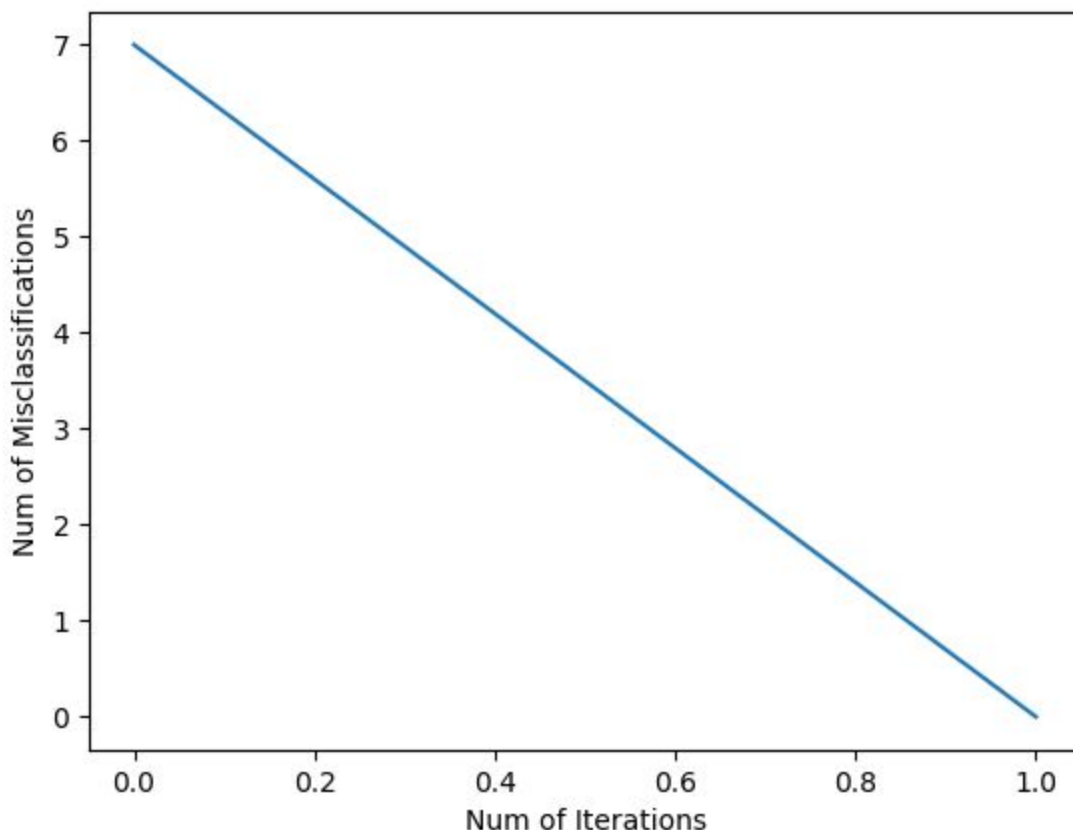


سوال ۱) پرسپترون

اول داده‌هایی که کلاسه‌شان ۲ است را دور می‌ریزیم و بعد آنها را به دو دسته‌ی ترین و تست تقسیم می‌کنیم (تا خط ۱۷)
 بعد با تابع `calcPhiMat` یک ستون ۱ به داده‌ها اضافه می‌کنیم (البته متوجه شدم که روی دقت تأثیری ندارد)
 پرسپترون بعد از حداکثر ۳ ایتريشن به وزن‌های بهینه‌ای همگرا می‌شود (ترین: خط ۳۶ تا ۶۲) و تقریباً در همه‌ی دفعات اجرا دقت بالای ۹۰ درصد دارد. (تست: خط ۶۸ تا ۷۳)
 در آخر هم از روی لیبل پیش‌بینی‌شده و لیبل اصلی، `confusion matrix` را می‌سازیم (خط ۷۶ تا ۸۰)



```
/usr/bin/python3.6 /home/esra/Desktop/pythonProject/HW5_prcptrn.py
Accuracy: 100.0
Num of epochs: 2
Confusion matrix (row indices are predicted values, column indices are real values)
    0    1
0  9.0  0.0
1  0.0 11.0
```

سوال ۲:

مشابه بالا اول داده‌ها را تقسیم می‌کنیم (تا خط ۱۷)
 بعد تابع `calcPhiMat` یک ستون ۱ به داده‌ها (x) اضافه می‌کند تا در ضریب بایاس ضرب شود، بعد تابع `calcTmat` کلاس‌های داده‌ها را به صورت one hot در می‌آورد، بعد تابع `calcPmat` برای هر داده برای هر کلاس عددی را نسبت می‌دهد که معیاری از احتمال تعلق آن داده به آن کلاس است (یعنی کلاسی که این عدد برای آن بزرگترین است پیش‌بینی رگرسور از کلاس داده است).

تابع اصلی `minimize_GD` است که رگرسور از آن استفاده می‌کند. تابع `cost_function` از روی پیش بینی‌های مدل، مقدار `loss` آن را حساب می‌کند. تابع `grad_cost_function` هم مشتق این تابع `loss` برحسب بردار وزن‌ها را حساب می‌کند (اینجا چون مسالهی کلاس‌بندی است، برای هر کلاس یک بردار وزن داریم که در داده ضرب می‌شود و معیاری از احتمال تعلق داده به کلاس نظیرش را برمی‌گرداند. این بردارها کنار یکدیگر در ماتریس `theta` هستند).

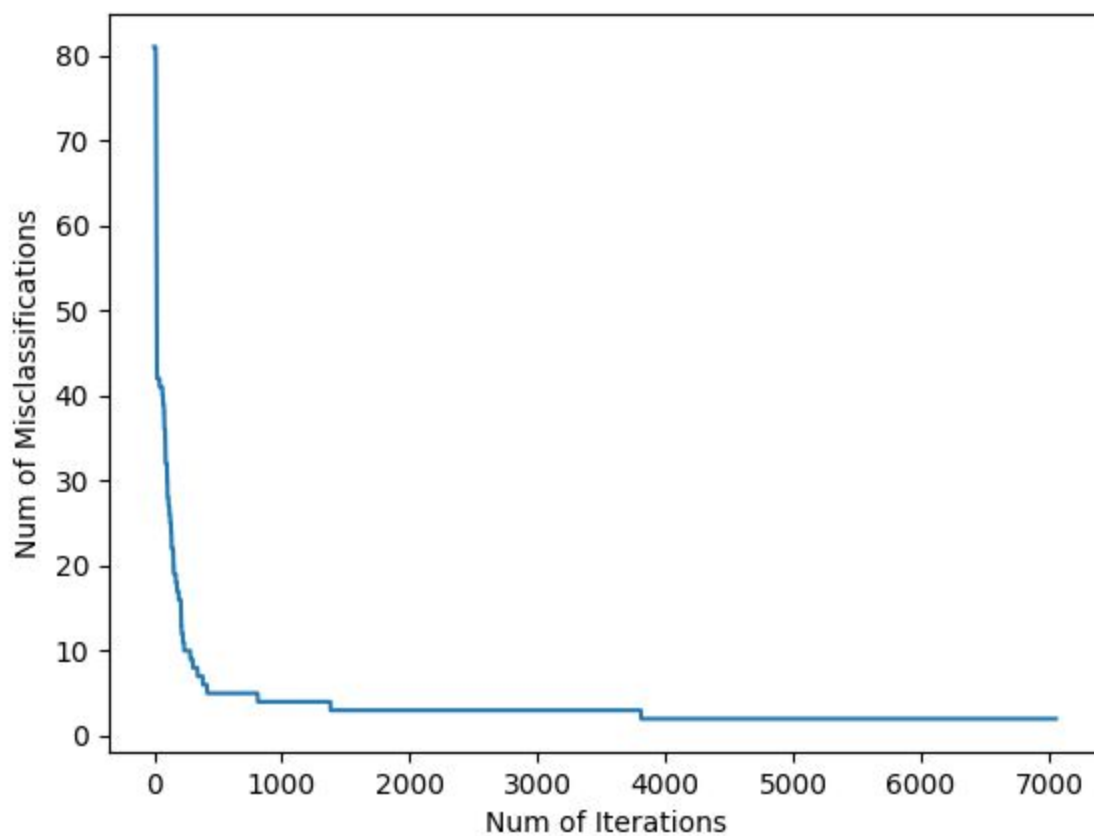
در `minimize_GD`، هر بار `x` (که اینجا همان `tetha` است که ابعادش برای ضرب عوض شده، هر بار از مشتق `cost function` نسبت به آن کم می‌شود (در واقع از ضریبی از آن کم می‌شود که این ضریب همان `learning rate` است) تا اینکه حداکثر ایتريشن‌های در نظر گرفته شده اجرا شود یا تغییرات `cost function` بین دو ایتريشن از حدی کمتر شود. هر بار هم تعداد داده‌هایی که غلط دسته‌بندی می‌شوند را در یک آرایه می‌ریزیم تا بعد آن را پلات کنیم و تغییرات دقت بر حسب ایتريشن را ببینیم.

برای تغییر لازمه در قسمت ۲، عبارت

`lam / (2.0 * N) * np.linalg.norm(thtvec) ** 2`

به تابع `cost function` اضافه می‌شود. به همین دلیل در مشتق آن نسبت به `theta` عبارت `lam / N * Theta` پیدا می‌شود. بدیهی است اگر `lam` را صفر بگیریم همان نتیجه‌ی قسمت ۱ بدست می‌آید.

قسمت ۱:

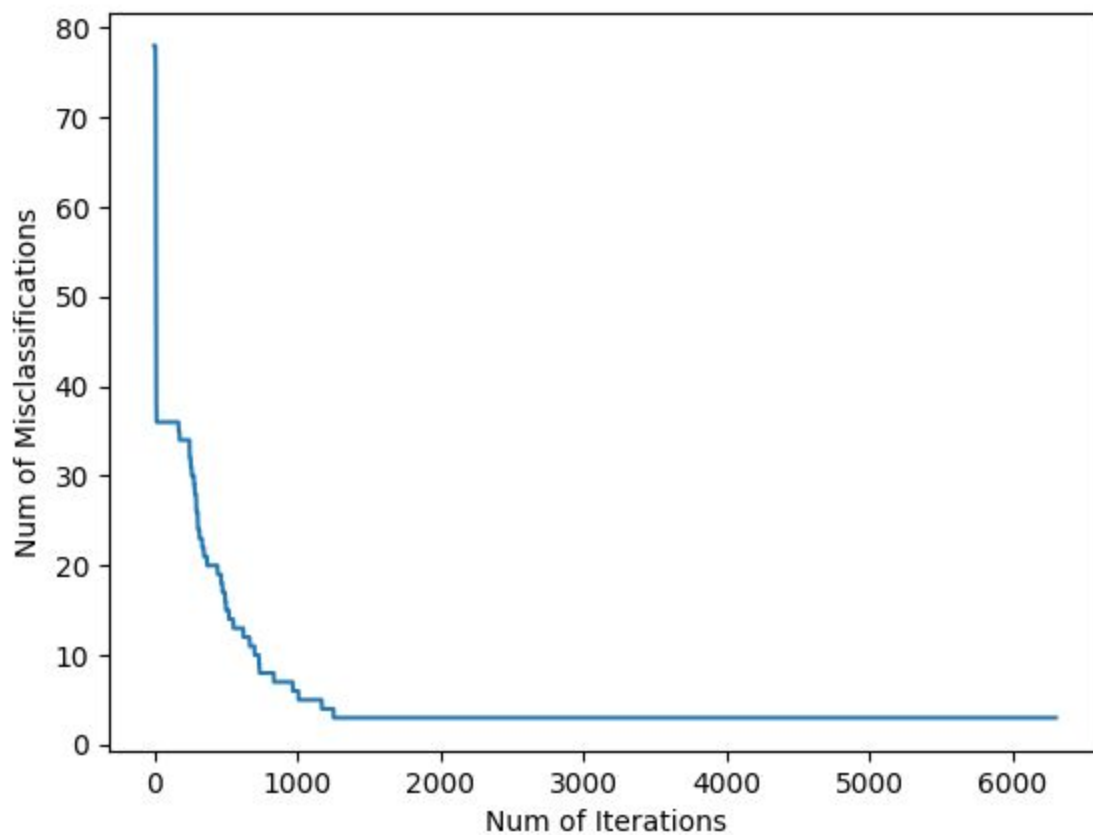


```
HW5_lgstc x
/usr/bin/python3.6 /home/esra/Desktop/pythonProject/HW5_lgstc
Accuracy: 96.66666666666667
Confusion matrix (row indices are predicted values, column in
      0      1      2
0  11.0   0.0   0.0
1   0.0   8.0   1.0
2   0.0   0.0  10.0

Process finished with exit code 0
```

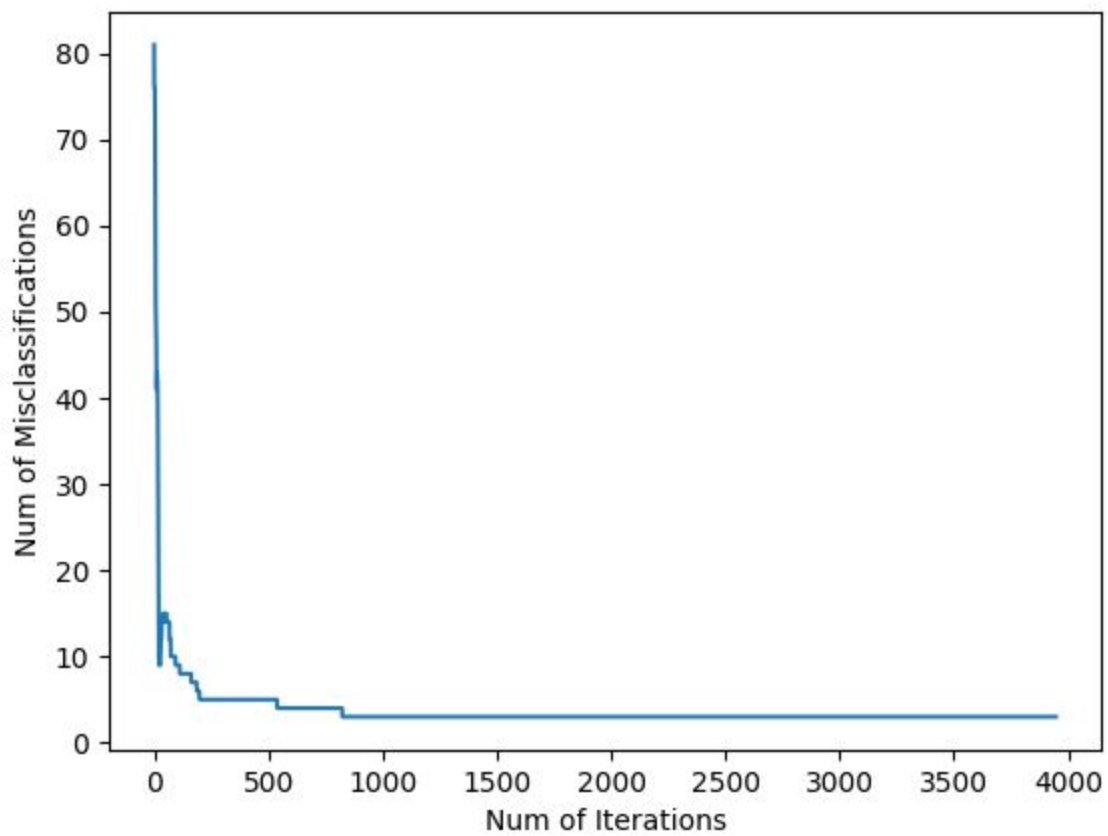
Lam = 0.1

قسمت ۲:



```
HW5_lgscc
/usr/bin/python3.6 /home/esra/Desktop/pythonProject/HW5_lgs
Accuracy: 96.66666666666667
Confusion matrix (row indices are predicted values, column
      0      1      2
0  8.0    0.0    0.0
1  0.0   13.0    0.0
2  0.0     1.0    8.0
```

Lam = 1

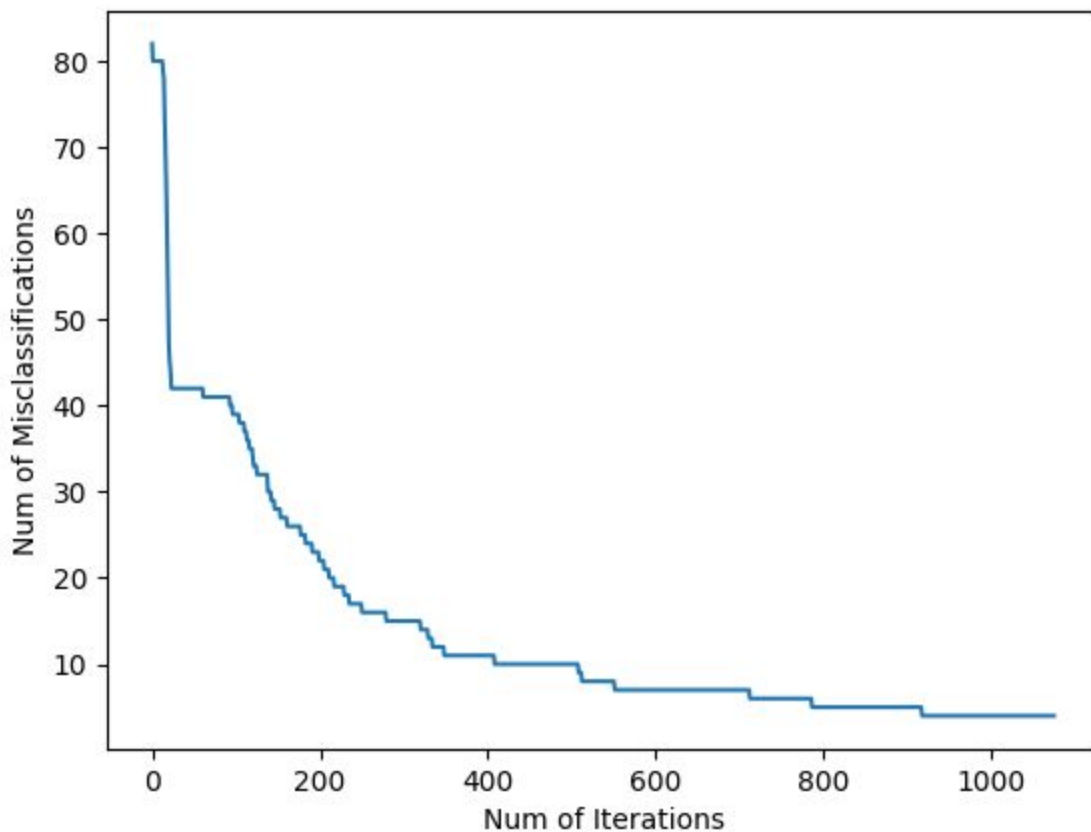


```

/usr/bin/python3.6 /home/esra/Desktop/pythonProject/HW5_lgstc
Accuracy: 100.0
Confusion matrix (row indices are predicted values, column in
    0    1    2
0  11.0  0.0  0.0
1   0.0  6.0  0.0
2   0.0  0.0 13.0

```

Lam = 10



```

/usr/bin/python3.6 /home/esra/Desktop/pythonProject/HW5_lgstc
Accuracy: 90.0
Confusion matrix (row indices are predicted values, column in
      0      1      2
0  10.0   0.0   0.0
1   0.0   6.0   0.0
2   0.0   3.0  11.0

Process finished with exit code 0

```

مشاهده می‌کنیم که در حالت آخر مدل بیش از حد برای اندازه‌ی بردار پنهانی در نظر می‌گیرد. هم دیرتر همگرا می‌شود هم اینکه خطا کمی بیشتر می‌شود که یعنی **underfit** شده. اگر درصد داده‌ی ترین را پایین‌تر می‌گرفتیم این اختلاف دقت مشهودتر بود. در کل به ازای $\text{lam}=1$ مدل زودتر از دو حالت دیگر همگرا می‌شود و به نظر می‌رسد این مقدار از دو مقدار دیگر مناسب‌تر باشد.